

Chapitre 3

Analyse Syntaxique

Théorie des Langages & Compilation

Dr Tegawendé F. BISSYANDÉ

Master Informatique – UFR/SEA
Université Joseph Ki-Zerbo



Plan du chapitre

- 1 Introduction
- 2 Grammaires non contextuelles
- 3 Analyse descendante et ascendante
- 4 Automate à pile et analyse ascendante
- 5 Conflits et ambiguïtés
- 6 Priorité et associativité des opérateurs
- 7 Outils d'analyse syntaxique
- 8 Compléments
- 9 Exercices

Objectif

Structurer une chaîne d'unités lexicales en unités syntaxiques (*syntagmes*).

Exemples selon le paradigme :

- *Langages impératifs* : variables, expressions, instructions, déclarations
- *Langages fonctionnels* : variables, expressions, définitions
- *Langages logiques* : variables, termes, buts, clauses

Résultat : souvent une représentation intermédiaire, en général sous forme d'**arbre syntaxique**.

Principe

La structure syntaxique des programmes peut être décrite par une **grammaire non contextuelle** (*context-free grammar*).

Les expressions régulières sont en général **insuffisantes** (cas des structures emboîtées).

Pour chaque grammaire non contextuelle, on peut construire un **automate à pile** (non déterministe) acceptant le langage décrit par cette grammaire.

Définition d'une grammaire

Définition

Une **grammaire** G est définie par un quadruplet (V_T, V_N, S, P) :

- V_T : alphabet (les **terminaux**)
- V_N : alphabet disjoint de V_T (les **non-terminaux**)
- $S \in V_N$: lettre appelée **axiome**
- P : ensemble fini de **règles de production**

Une grammaire est dite **non contextuelle** (ou algébrique) si :

$$P \subset V_N \times (V_T \cup V_N)^*$$

c'est-à-dire qu'il n'y a qu'un **seul non-terminal** en partie gauche de chaque règle.

Exemple : grammaire des expressions arithmétiques

$$E ::= E + E \mid E - E \mid E * E \mid E / E \mid (E) \mid \text{int}$$

- $V_T = \{+, -, *, /, (,), \text{int}\}$
- $V_N = \{E\}$
- $S = E$
- 6 règles de production

Problème : cette grammaire est **ambiguë** (l'expression $1+2*3$ admet plusieurs arbres syntaxiques).

Langage engendré

Le langage $L(G)$ engendré par une grammaire G est l'ensemble des mots produits en partant du symbole de départ S et en appliquant des règles de production.

Dérivation :

- 1 Si α n'est formé que de terminaux, alors α est un mot ω de $L(G)$
- 2 Sinon, α peut se découper en $\beta A \gamma$, où A est un non-terminal
- 3 On considère une production $A ::= \delta$, on remplace A par δ , noté $\alpha \Rightarrow \beta \delta \gamma$

Définition

Un arbre de dérivation syntaxique pour la grammaire G de racine $X \in V_N$ et de feuilles $\omega \in V_T^*$ est un arbre ordonné dont :

- La racine est X
- Les feuilles sont étiquetées par des terminaux formant le mot ω
- Les nœuds internes sont des non-terminaux
- Si Y est un nœud interne dont les fils sont étiquetés $\alpha_1 \dots \alpha_p$, alors $Y ::= \alpha_1 \dots \alpha_p$ est une production de P

Définition

Une phrase $x \in L(G)$ est **ambiguë** si elle admet **plusieurs arbres syntaxiques** différents.

Une grammaire est dite ambiguë si elle engendre au moins un mot ambigu.

Exemple : l'expression $1 + 2 * 3$ avec la grammaire $E ::= E + E \mid E * E \mid \text{int}$ admet deux arbres selon qu'on associe $+$ ou $*$ en premier.

Solutions :

- Rendre la grammaire non ambiguë (introduire des niveaux de priorité)
- Utiliser des règles de **priorité** et d'**associativité**

Analyse descendante vs. ascendante

Définition

Analyser un mot, c'est établir si le mot appartient au langage engendré par la grammaire. En pratique, on construit une dérivation ou un arbre de dérivation syntaxique.

Analyse descendante

Part du symbole de départ (axiome) et l'**expanse** jusqu'à obtenir le mot.

Familles : LL(1), LL(k), analyse prédictive.

Analyse ascendante

Factorise le mot en reconnaissant des parties droites de production jusqu'à retomber sur l'axiome.

Familles : LR(0), SLR, LALR, LR(1).

Tout comme les langages réguliers sont reconnus par des automates finis, les langages non contextuels (algébriques) sont reconnus par des **automates à pile**.

La pile permet de **mémoriser des informations** au cours des transformations pour décider de la prochaine transition à effectuer. Cette pile est non bornée et peut stocker des informations de taille variable.

Principe

On part du texte source (lecture de gauche à droite) et on cherche à remonter jusqu'à l'axiome par des actions de **lecture** ou de **réduction**.

- **Lecture** (*shift*) : consommer et empiler un lexème
- **Réduction** (*reduce*) : reconnaître la partie droite d'une production au sommet de la pile et la transformer en le non-terminal correspondant

Évolution :

- État initial : pile vide
- État final : entrée totalement lue, pile contient uniquement $S \Rightarrow$ **accepté**

Exemple d'analyse ascendante

Grammaire : $G_0 = \langle \{a, b, c\}, \{S, X, Y\}, \{S\}, \{S ::= XY, X ::= aX \mid c, Y ::= b\} \rangle$

Analyse du mot acb :

Pile	Entrée	Action	Représentation
	acb	shift	$.acb$
a	cb	shift	$a.cb$
ac	b	reduce $X ::= c$	$ac.b$
aX	b	reduce $X ::= aX$	$aX.b$
X	b	shift	$X.b$
Xb		reduce $Y ::= b$	$Xb.$
XY		reduce $S ::= XY$	$XY.$
S		accept	$S.$

Conflit reduce-reduce

Deux réductions possibles pour le même sommet de pile.

Exemple : $E ::= T \mid \text{Id}$ et $T ::= \text{Id} \Rightarrow$ si Id est dans la pile, deux réductions possibles.

Conflit shift-reduce

Le sommet de pile correspond à une partie droite complète, mais on pourrait aussi lire le prochain token.

Exemple : $E ::= E + E \mid E * E \mid \text{Id}$, avec la pile $E + E$ et le prochain token $*$.

Conflit shift-reduce classique :

$$stmt ::= IF\ expr\ stmt \mid IF\ expr\ stmt\ ELSE\ stmt$$

Avec l'exemple : $IF\ e_1\ IF\ e_2\ s_1 \bullet\ ELSE\ s_2$

Deux choix possibles :

- ❶ **Shift** ELSE : le ELSE est associé au IF le plus proche (solution standard)
- ❷ **Reduce** $IF\ e_2\ s_1$: le ELSE est associé au premier IF

La première solution (**shift**) est communément utilisée dans les langages de programmation modernes et correspond au comportement par défaut de la plupart des générateurs d'analyseurs.

Priorité et associativité

Les générateurs d'analyseurs syntaxiques permettent de spécifier :

- **Associativité** : %left, %right, %nonassoc
- **Priorité** : définie par l'**ordre** des déclarations (du moins prioritaire au plus prioritaire)

Exemple (Ocamlyacc / Bison) :

```
%token <int> INT
%left  PLUS MINUS      /* priorite la plus faible */
%left  TIMES DIV        /* priorite moyenne      */
%nonassoc UMINUS        /* priorite la plus forte  */
```

%prec : permet de modifier la priorité d'une règle particulière.

```
expr: MINUS expr %prec UMINUS { - $2 }
```


Élimination du conflit if-else

Méthode 1 : directive du générateur

```
%nonassoc IF_LOWER_ELSE
%nonassoc ELSE

stmt:
    IF expr stmt %prec IF_LOWER_ELSE
  | IF expr stmt ELSE stmt
```

Méthode 2 : réécriture de la grammaire

$$\begin{aligned} \text{stmt} &::= \text{stmt-complete} \mid \text{stmt-incomplete} \\ \text{stmt-complete} &::= \text{IF expr stmt-complete ELSE stmt-complete} \mid \text{autre} \\ \text{stmt-incomplete} &::= \text{IF expr stmt} \mid \text{IF expr stmt-complete ELSE stmt-incomplete} \end{aligned}$$

Structure du fichier source :

```
%{  
    Prelude (code OCaml/C)  
%}  
Declarations (tokens, priorites)  
%%  
Regles (productions + actions semantiques)  
%%  
Epilogue (code OCaml/C)
```

Déclarations : %token, %left, %right, %nonassoc, %start, %type

Actions sémantiques : code entre accolades associé à chaque règle, utilisant \$1, \$2, ... pour référencer les valeurs des symboles.

Exemple complet : calculatrice

```
%{ let vars = Hashtbl.create 17 %}  
%token <int> INT  
%token <string> IDENT  
%token PLUS MINUS TIMES DIV  
%token LPAREN RPAREN SEMICOLON EQUAL VAR EOF  
%left PLUS MINUS  
%left TIMES DIV  
%nonassoc UMINUS  
%start calcul  
%type <int> calcul  
%%  
calcul:  
    declarations expr EOF    { $2 }  
  | expr EOF                { $1 }  
;  
expr:  
    expr PLUS expr           { $1 + $3 }  
  | expr MINUS expr          { $1 - $3 }  
  | expr TIMES expr          { $1 * $3 }  
  | expr DIV expr            { $1 / $3 }  
  | MINUS expr %prec UMINUS  { - $2 }  
  | LPAREN expr RPAREN       { $2 }  
  | INT                      { $1 }  
  | IDENT                    { Hashtbl.find vars $1 }  
;
```

Grammaires récursives droite et gauche

Récursivité gauche

$$list ::= id \mid list, id$$

Bien adaptée à l'analyse ascendante (LR).

Récursivité droite

$$list ::= id \mid id, list$$

Bien adaptée à l'analyse descendante (LL).

Remarque : les analyseurs LR (Bison, Ocamlyacc) préfèrent la récursivité gauche car elle utilise une pile de taille bornée. Les analyseurs LL préfèrent la récursivité droite pour éviter la boucle infinie.

Principe

L'analyse descendante prédictive utilise un seul symbole de prévision (*lookahead*) pour décider quelle production appliquer, **sans retour en arrière**.

Ensembles FIRST et FOLLOW :

- $\text{FIRST}(\alpha)$: ensemble des terminaux qui peuvent commencer une chaîne dérivée de α
- $\text{FOLLOW}(A)$: ensemble des terminaux qui peuvent apparaître immédiatement après A dans une dérivation

Condition LL(1) : pour deux productions $A ::= \alpha$ et $A ::= \beta$:

$$\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$$

Si $\alpha \Rightarrow^* \varepsilon$, alors $\text{FIRST}(\beta) \cap \text{FOLLOW}(A) = \emptyset$.

Table d'analyse LL(1)

Construction

Pour chaque production $A ::= \alpha$:

- 1 Pour chaque terminal $a \in \text{FIRST}(\alpha)$, ajouter $A ::= \alpha$ dans $M[A, a]$
- 2 Si $\varepsilon \in \text{FIRST}(\alpha)$, pour chaque $b \in \text{FOLLOW}(A)$, ajouter $A ::= \alpha$ dans $M[A, b]$

Exemple : $E ::= TE'$, $E' ::= +TE' \mid \varepsilon$, $T ::= FT'$, $T' ::= *FT' \mid \varepsilon$, $F ::= (E) \mid \text{id}$

	id	+	*	(	)	\$
E	TE'			TE'		
E'		$+TE'$			ε	ε
T	FT'			FT'		
T'		ε	$*FT'$		ε	ε
F	id			(E)		

Élimination de la récursivité gauche

Les grammaires LL ne supportent pas la récursivité gauche. Il faut la transformer :

Règle générale : si $A ::= A\alpha \mid \beta$ (avec β ne commençant pas par A), remplacer par :

$$A ::= \beta A' \quad A' ::= \alpha A' \mid \varepsilon$$

Exemple :

Avant :

$$E ::= E + T \mid T$$

Après :

$$\begin{aligned} E &::= T E' \\ E' &::= + T E' \mid \varepsilon \end{aligned}$$

Familles d'analyseurs LR

L'analyse ascendante LR lit l'entrée de **gauche à droite** (L) et construit une dérivation **droite** en sens inverse (R).

Type	Description	Puissance
LR(0)	Pas de lookahead	Faible
SLR(1)	Utilise FOLLOW pour résoudre	Modérée
LALR(1)	Fusion d'états LR(1)	Standard (Yacc/Bison)
LR(1)	Lookahead complet par état	Maximale

Inclusion : $LR(0) \subsetneq SLR(1) \subsetneq LALR(1) \subsetneq LR(1)$

En pratique, **LALR(1)** (utilisé par Yacc, Bison, Ocamlyacc) offre le meilleur compromis entre puissance et taille des tables.

Forme normale de Backus-Naur (BNF)

Définition

La notation BNF (*Backus-Naur Form*) est une métasyntaxe utilisée pour décrire la grammaire des langages de programmation.

Conventions :

- $\langle \text{non-terminal} \rangle$: symboles entre chevrons
- $::=$: séparateur entre partie gauche et droite
- $|$: alternative entre productions

EBNF (Extended BNF, norme ISO 14977) : ajoute :

- $[\dots]$: élément optionnel (0 ou 1 fois)
- $\{ \dots \}$: répétition (0 ou n fois)
- $(\dots)$: groupement

Utilisée dans les spécifications de C, Java, Python, SQL, XML, JSON, ...

Exercice 1 – Grammaires

- 1 Soit la grammaire : $S ::= aSb \mid ab$. Quel est le langage engendré ?
- 2 Soit la grammaire : $S ::= aB \mid bA$, $A ::= a \mid aS \mid bAA$, $B ::= b \mid bS \mid aBB$. Montrer que le mot $aabbab$ appartient à $L(G)$ en donnant une dérivation.
- 3 Donner une grammaire non contextuelle pour le langage des expressions booléennes avec les opérateurs AND, OR, NOT et les constantes `true`, `false`, en respectant les priorités habituelles.

Exercice 2 – Arbres de dérivation

Soit la grammaire :

$$E ::= E + T \mid T \quad T ::= T * F \mid F \quad F ::= (E) \mid \text{id}$$

- 1 Dessiner l'arbre de dérivation pour $\text{id} + \text{id} * \text{id}$.
- 2 Montrer que cette grammaire n'est **pas ambiguë** (contrairement à $E ::= E + E \mid E * E \mid \text{id}$).
- 3 Effectuer l'analyse ascendante (shift-reduce) du mot $\text{id} * \text{id} + \text{id}$ en montrant l'évolution de la pile à chaque étape.

Exercice 3 – Analyse ascendante

Soit la grammaire : $S ::= aABe$, $A ::= Abc \mid b$, $B ::= d$

- 1 Effectuer l'analyse ascendante du mot `abbcde` en montrant à chaque étape : la pile, l'entrée restante, et l'action (shift ou reduce).
- 2 Le mot `abbcde` appartient-il à $L(G)$? Justifier.

Exercice 4 : Donner les déclarations Ocamlyacc/Bison nécessaires pour une grammaire d'un langage simple avec :

- Affectation : `id = expr ;`
- Expressions arithmétiques avec `+`, `-`, `*`, `/` et moins unaire
- Conditionnelle : `if expr then stmt else stmt`
- Boucle : `while expr do stmt`

Préciser les priorités et associativités.